FIT5217 - Assignment 2

Marks	Worth 100 marks, and 25% of all marks for the unit
Due Date	Week 11 Lecture Date, 11:55 PM
Extension	An extension could be granted under some circumstances. A special consideration application form must be submitted. Please refer to the university webpage on special consideration.
Lateness	For all assessment items handed in after the official due date, and without an agreed extension, a 5% penalty applies to the student's mark for each day after the due date (including weekends) for up to 7 days. Assessment items handed in after 7 days without special consideration will not be considered.
Authorship	This is an individual assessment. All work must be your own. All submissions will be placed through Turnitin. This makes plagiarism remarkably easy to identify for us.
Submission	5 files: a Jupyter notebook file, a pdf print of the Jupyter notebook, an XLSX file, a PPTX file, and a 15-minute video presentation need to be submitted. The name of the files must be `code_012345678.ipynb`, `code_012345678.pdf`, `generated_012345678.xlsx`, `presentation_012345678.pptx`, `recording_012345678.mp4` where "`012345678`" is replaced by your own student ID. There will be a 5% penalty for every missing file in the submission.

Table 1: Instructions for FIT5217 - Assignment 2

Introduction 🥘

Came back home after a long day, opened your fridge, you got a few ingredients in there, but have no clue how to cook something with them. You reach out for your 📞 and call your mom, but she does not answer your call. She had enough of you already! You reach out for your 📱, search for a cooking App, but surprisingly there is nothing in the App Store. Well, why not build one yourself? You could train a deep neural model to do the job for you. This is simply a Sequence-to-Sequence modelling task, mapping ingredients to a recipe.

Optional Literature Review 📝

No real research project could be done without a proper literature review. Here are some optional readings for you.

Paper: `Kiddon, C., Zettlemoyer, L., Choi, Y. (2016) Globally Coherent Text Generation with Neural Checklist Models. In Proceedings of the Conference on Empirical Methods in Natural Language Processing.`

Paper: `See, A., Liu, P., Manning, C. Get To The Point: Summarization with Pointer-Generator Networks. (2017) In Proceedings of the Annual Meeting of the Association for Computational Linguistics.`

Paper: `Lu, X., West, P., Zellers, R., Le Bras, R., Bhagavatula, C., Choi, Y. (2021) NeuroLogic Decoding: (Un)supervised Neural Text Generation with Predicate Logic Constraints. In Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics.`

Paper: `Tu, Z., Zhengdong, L., Yang L. Xiaohua, L., Hang, Li. (2016) Modeling Coverage for Neural Machine Translation. In Proceedings of the Annual Meeting of the Association for Computational Linguistics.`

Paper: `Welleck, S., Kulikov, I., Roller, S., Dinan, E., Cho, K., Weston, J. (2020) Neural text degeneration with unlikelihood training. In Proceedings of the International Conference on Learning Representations.`

Paper: `Vinyals, O. Bengio, S., Kudlur, M. (2016) ORDER MATTERS: SEQUENCE TO SEQUENCE FOR SETS. In Proceedings of the International Conference on Learning Representations.`

Neural Chef Assistant 🤖 (Total 80+20 Marks)

The task is to train a deep neural model for recipe generation. The dataset we will use is from the Kiddon et al paper you (hopefully) reviewed above. Download the dataset from Moodle. You should use PyTorch for this part (need CE's approval if you want to use a different deep learning library). The test data should not be used for training and it should not be pre-processed or adjusted at any level. For each sample only use the ingredients and recipes, and ignore the titles. A sample from the data is demonstrated in Table 2.

Title: Apple Crisp
Ingredients: `["8 c. sliced apples", "8 tbsp. flour", "3 c. sugar", "2 tsp. cinnamon"]`
Recipe: `["Preheat oven to 350 degrees.", "Mix ingredients together and put into a 9 X 13-inch greased pan."]`
Title: Mexican Lasagna
Ingredients: `["1 1/2 lb. beef, browned", "1 can Ro-Tel", "1 can tomato sauce", "1 tsp. sugar", "1 tsp. chili powder", "1/4 tsp. pepper", "1 tsp. basil", "1 box lasagna noodles", "3 oz. cream cheese", "8 oz. sour cream", "Cheddar cheese"]`
Recipe: `["Cook first 7 ingredients for 20 minutes.", "Cook lasagna noodles.", "Mix cream cheese and sour cream together.", "Layer lasagna noodles, meat mixture, then cheese sauce.", "Should make 2 layers, then top with Cheddar cheese."]`

Table 2: A sample from data.

1 Configurations

We want you to build 6 Encoder-Decoder models. You should NOT be opting for any backbone other than the RNN family (i.e., cannot use CNN, Transformer, or GNN).

1.1 The 2 Baselines🌶️

Start from the tutorial code in Week 7 and revise it for this task and build your two baseline models.

- Baseline 1: Sequence-to-Sequence model without attention [4 Marks]
- Baseline 2: Sequence-to-Sequence model with attention [5 Marks]

1.2 The 2 Mild Extensions🌶️🌶️

Pick the baseline that works better and extend it by building 2 other models ([8 Marks] each) by choosing to do any of the followings (or a combination of them):

- Preprocessing the data beyond simple lowercasing/stop-word removal/lemmatization/stemming (i.e., removing measurement units in the ingredients list, padding recipe sentences, vocabulary pruning, etc).

- Changing basic model configurations (i.e., dimensionality of hidden states or embeddings, opting for a different member of the RNN family), or training configurations (i.e., Teacher Force Rating, batch size, maximum length, etc).
- Using pretrained embeddings (i.e., word2vec, GLoVe)
- Separating or sharing the embedding matrices for encoder and decoder
- Stacking up more layers in the encoder or decoder
- Pre-training the encoder and the decoder separately (i.e., using auto-encoding, etc), and re-using the pretrained encoder and decoder as the starting point of the training rather than starting from random initialisation of weights.

1.3 The 2 Spicy Extensions 🌶🌶🌶🌶🌶

Now build another 2 models ([10 Marks] each) by choosing to do any of the followings (or a combination of them):

- Implementing a copy mechanism (i.e., read 2nd reference under the Optional Review)
- Implementing a coverage mechanism (i.e., have a look at 1st, 3rd, 4th, and 5th references under optional review).
- Implementing a different input encoding technique to process the ingredients as a set rather than a list (i.e., have a look at the 6th reference under optional review)

The idea is that only those of you who put in an extra work could collect these spicy 20 marks. We will not be marking for the effort (unfortunately) if you only have a partial implementation but no results.

2 Performance

While achieving a specific performance threshold is not required, you are expected to make informed and well-justified design decisions in your model extensions. Your goal should be to explore modifications that have the potential to improve performance relative to the baselines. We expect to see that there is at least 10% of performance improvement (e.g., improving BLEU from 10 to 11) on the test set between the baseline models and the best extension model you have implemented. Ideally, for your design choices you should only be looking at the training and dev sets. [5 Marks]

3 Jupyter notebook

Do not remove output logs from the notebook you submit. Keep all the training and test outputs. The Jupyter notebook you submit should contain all the code, logs, and outputs needed to fill different sections of your presentation. Make sure your notebook has clear markdowns that are easy to navigate and locate which part of the notebook does what, etc. [5 Marks]

All models should sit within 1 notebook file `code_012345678.ipynb`. But if you end up with 1 notebook per model, submit multiple notebooks but name them as `code_012345678_base1.ipynb`, `code_012345678_base2.ipynb`, `code_012345678_mild1.ipynb`, `code_012345678_mild2.ipynb`, `code_012345678_spicy1.ipynb`, `code_012345678_spicy2.ipynb`. **However the pdf print of**

these notebooks should still be merged into a single pdf file `code_012345678.pdf` and it should be clear which parts of the pdf correspond to which model implementation.

4 XLSX

All generated outputs from the 4-6 models should be reported in the corresponding column of the generated_012345678.xlsx file you submit for this assignment. For this, you are provided with the template XLSX file, where each row has 7 columns: Ingredients list, followed by 6 other columns (i.e., Baseline 1, 2, etc), each recording the generated recipe from the corresponding model. If you have not implemented any of the models, just leave the corresponding cell empty. Do not change the order of the rows, and do not remove and add any row.

5 Presentation - PPTX and Recording

Check the PPTX template to understand what goes under each of these headings in the PPTX. The presentation recording should follow the instructions outlined on the front page of the PPTX. You need to clealry cover each section in your recording (maximum 15 minutes).

Abstract [5 Marks]

Data Statistics [5 Marks]

Models and Training Configurations [10 Marks]

Training Analysis [10 Marks]

Quantitative Comparison [10 Marks]

Qualitative Comparison [5 Marks]

6 Evaluation Metrics

You can use the following code for measuring the evaluation metrics (BLEU-4, METEOR, BertScore). Obviously, this is only for 1 example so when you are reporting these numbers you need to measure them over the entire test set and report the average score.

```
pip install nltk bert-score

import nltk
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
from nltk.translate.meteor_score import meteor_score
from bert_score import score

# Download required NLTK resources
nltk.download('punkt_tab')      # for word_tokenize
nltk.download('wordnet')        # for METEOR
```

```
nltk.download('omw-1.4')       # also needed for METEOR

# Toy Input texts (gold and predicted recipes)
gold = """cream butter and sugar . add egg yolks , one at a time and beat until mixture is very light and lemon colored . add vanilla and milk . in another bowl beat egg whites until foamy . fold into creamed mixture . fold in 11/2 cups of the coconut . pour into baked pie shell and sprinkle with nutmeg . top with remaining grated coconut . bake in preheated 350 oven for about 35 minutes or until filling is just set . serve warm or cold ."""

pred = """45-50 minutes coconut custard pie preheat oven to 350°f . in a mixing bowl , cream butter and confectioners sugar until light and fluffy . add egg yolks one at a time , beating well after each addition . stir in vanilla , evaporated milk , and salt ; mix until smooth . fold in grated coconut . in a separate bowl , beat egg whites until stiff peaks form ; gently fold into coconut mixture . pour filling into the baked 9-inch pie shell and smooth the top . sprinkle with ground nutmeg . bake for 30 to 35 minutes or until set and lightly golden . cool completely before serving . chill for cleaner slices ."""

# Tokenize for BLEU
ref_tokens = nltk.word_tokenize(gold)
hyp_tokens = nltk.word_tokenize(pred)

# BLEU-4 with smoothing
smoothie = SmoothingFunction().method4
bleu4 = sentence_bleu([ref_tokens], hyp_tokens, weights=(0.25, 0.25, 0.25, 0.25), smoothing_function=smoothie)

# METEOR - requires list of tokenized references and hypothesis
meteor = meteor_score([ref_tokens], hyp_tokens)

# BERTScore
P, R, F1 = score([pred], [gold], lang='en', verbose=False)
bertscore_f1 = F1.numpy().item()  # Convert tensor to numpy then Python scalar

# Print results
print(f"BLEU-4 Score: {bleu4:.4f}")
print(f"METEOR Score: {meteor:.4f}")
print(f"BERTScore (F1): {bertscore_f1:.4f}")

# Output printed
BLEU-4 Score: 0.1470
METEOR Score: 0.5213
BERTScore (F1): 0.8890
```